Minimal Spanning Trees in Medical Image Processing

Application Level Assignment Report

Computational Mathematics for Engineers (Minor)

Submitted by

Arya Vinod

B.Tech Computer Science and Engineering
Saintgits College of Engineering

Abstract

Medical image segmentation plays an important role in medical diagnosis and treatment planning. This report presents an implementation of a Minimal Spanning Tree (MST) based segmentation algorithm inspired by Kruskal's algorithm. The image is represented as a graph where pixels are treated as nodes and edges represent the relationship between neighboring pixels. The algorithm groups similar pixels to form meaningful segments while reducing noise. The segmentation result helps in identifying important structures in medical images.

1 Introduction

Medical images such as MRI, CT scans and X-rays are widely used for diagnosis in healthcare. However, analyzing these images manually is often difficult due to noise and unclear boundaries.

Image segmentation helps to divide an image into meaningful regions. Graph based image segmentation methods provide an efficient way to perform this task.

In this project, we implement an MST based segmentation method inspired by Kruskal's algorithm to group similar pixels into segments.

2 Methodology

The methodology follows a graph-theoretic approach where each MRI scan is modeled as a grid graph $G = (V, E)$, establishing 4-neighbor connectivity between pixel nodes with edge weights defined by the intensity gradient $w(v_i, v_j) = |I(v_i) - I(v_j)|$. Using a Union-Find data structure with path compression, the algorithm executes a greedy Kruskal-based merging process, joining components C_1 and C_2 only if the connecting edge weight remains below a dynamic threshold determined by the parameter k and the segment size $|C|$. Following segmentation, an adaptive denoising pass is performed by replacing every pixel's intensity with the mean value of its respective MST component, effectively smoothing internal noise while preserving sharp anatomical boundaries. The process concludes with a quantitative evaluation using MSE and PSNR metrics, alongside a log-scale Segment Distribution Histogram to statistically verify the granularity of the resulting anatomical clusters.

2.1 Graph Representation

An image can be represented as a graph

$$G = (V, E)$$

where

- V represents the pixels
- E represents edges between neighboring pixels

The weight of an edge is defined as

$$w(v_{i,j}, v_{k,l}) = |I(i,j) - I(k,l)|$$

where $I(i,j)$ is the intensity of the pixel.

2.2 Segmentation Algorithm

The segmentation process follows these steps:

1. Each pixel is treated as an individual component.
2. Edges are created between neighboring pixels.
3. Edges are sorted based on weight.
4. Components are merged using a threshold condition.

2.3 Segmentation Criterion

Two components are merged if

$$w(e) \leq min(Int(C_u) + \tau(C_u), Int(C_v) + \tau(C_v))$$

where

$$\tau(C) = \frac{k}{|C|}$$

2.4 Denoising

After segmentation, denoising is performed by averaging pixel intensities within each segment.

$$I_{denoised}(p) = \frac{1}{|C(p)|} \sum_{q \in C(p)} I(q)$$

3 Implementation

The MST-based image segmentation algorithm was implemented using Python. The implementation includes image preprocessing, graph construction, segmentation using a modified Kruskal's algorithm, and denoising.

First, the input medical image is loaded and converted to grayscale if necessary. The pixel intensity values are normalized so that intensity differences between neighboring pixels can be calculated easily.

Next, the image is represented as a graph where each pixel acts as a vertex. Edges are created between neighboring pixels using a 4-neighborhood model. The weight of each edge is calculated as the absolute difference between the intensity values of the connected pixels.

The segmentation process is performed using a modified version of Kruskal's algorithm. Initially, each pixel is treated as a separate component. Edges are processed in

increasing order of weight, and components are merged if the merging condition based on the threshold criterion is satisfied. A Union–Find data structure is used to efficiently manage the connected components during this process.

After segmentation, denoising is applied by replacing each pixel value with the average intensity of the pixels in the same segment. Finally, the results are visualized to compare the original image, segmented image, and denoised image.

The following libraries were used:

- NumPy
- Matplotlib
- Scikit-image
- time
- scipy
- math
- os

4 Python Implementation

```
import numpy as np
import matplotlib.pyplot as plt
from skimage import io
from collections import defaultdict
import time
import math
import os

# Ensures plots render correctly in Google Colab environments
%matplotlib inline

# ==========================================
# 1. UNION-FIND DATA STRUCTURE
# ==========================================
class UnionFind:
    def __init__(self, size):
        self.parent = np.arange(size)
        self.size = np.ones(size)
        self.max_weight = np.zeros(size)

    def find(self, i):
        if self.parent[i] == i:
            return i
        self.parent[i] = self.find(self.parent[i]) # Path
           compression
        return self.parent[i]
```

```
    def union(self, i, j, weight):
        root_i = self.find(i)
        root_j = self.find(j)
        if root_i != root_j:
            if self.size[root_i] < self.size[root_j]:
                root_i, root_j = root_j, root_i
            self.parent[root_j] = root_i
            self.size[root_i] += self.size[root_j]
            # Update Int(C) as max weight edge in MST
            self.max_weight[root_i] = max(self.max_weight[root_i
               ], weight)

# ==========================================
# 2. GRAPH CONSTRUCTION & SEGMENTATION
# ==========================================
def build_graph(image):
    rows, cols = image.shape
    edges = []
    # Connect 4-neighbor pixels
    for i in range(rows):
        for j in range(cols):
            u = i * cols + j
            if j + 1 < cols:
                v = i * cols + (j + 1)
                weight = abs(float(image[i, j]) - float(image[i,
                   j + 1]))
                edges.append((weight, u, v))
            if i + 1 < rows:
                v = (i + 1) * cols + j
                weight = abs(float(image[i, j]) - float(image[i +
                    1, j]))
                edges.append((weight, u, v))
    return edges

def segment_image(image, k):
    rows, cols = image.shape
    num_pixels = rows * cols
    edges = build_graph(image)
    edges.sort(key=lambda x: x[0])
    uf = UnionFind(num_pixels)

    for weight, u, v in edges:
        root_u = uf.find(u)
        root_v = uf.find(v)
        if root_u != root_v:
            # tau(C) = k / |C|
            tau_u = k / uf.size[root_u]
            tau_v = k / uf.size[root_v]
            # Merging criterion: weight <= min(Int + tau)
```

```
            if weight <= min(uf.max_weight[root_u] + tau_u, uf.
               max_weight[root_v] + tau_v):
                uf.union(root_u, root_v, weight)
    return uf

# ==========================================
# 3. DENOISING & LABEL EXTRACTION
# ==========================================
def process_output(image, uf):
    rows, cols = image.shape
    denoised = np.zeros_like(image, dtype=np.float64)
    labels = np.zeros((rows, cols), dtype=np.int32)
    segment_sums = defaultdict(float)
    segment_sizes = {}

    root_to_id = {}
    next_id = 0

    # Pass 1: Sum intensities and map root pointers to small IDs
       (0...N)
    for i in range(rows):
        for j in range(cols):
            pixel_idx = i * cols + j
            root = uf.find(pixel_idx)
            if root not in root_to_id:
                root_to_id[root] = next_id
                next_id += 1
                segment_sizes[root] = uf.size[root]
            labels[i, j] = root_to_id[root]
            segment_sums[root] += float(image[i, j])

    # Pass 2: Calculate Mean Intensity for Denoised result
    for i in range(rows):
        for j in range(cols):
            root = uf.find(i * cols + j)
            denoised[i, j] = segment_sums[root] / uf.size[root]

    return denoised, labels, list(segment_sizes.values())

# ==========================================
# 4. METRICS CALCULATION
# ==========================================
def calculate_metrics(original, denoised):
    mse = np.mean((original.astype(np.float64) - denoised) ** 2)
    psnr = 100 if mse == 0 else 20 * math.log10(255.0 / math.sqrt
       (mse))
    return psnr, mse

# ==========================================
# 5. VISUALIZATION & SEGMENT DISTRIBUTION HISTOGRAM
# ==========================================
```

```
def visualize_results(original, labels, denoised, sizes, filename
    , psnr):
    fig, axs = plt.subplots(1, 4, figsize=(24, 6))

    # Original Image
    axs[0].imshow(original, cmap='gray')
    axs[0].set_title(f'Original: {filename}')
    axs[0].axis('off')


    axs[1].imshow(labels, cmap='nipy_spectral')
    axs[1].set_title('Segmentation Boundaries')
    axs[1].axis('off')

    # Denoised Grayscale Image
    axs[2].imshow(denoised, cmap='gray')
    axs[2].set_title(f'Denoised (PSNR: {psnr:.2f}dB)')
    axs[2].axis('off')

    # Segment Size Distribution Histogram
    axs[3].hist(sizes, bins=50, color='forestgreen', edgecolor='
       black', log=True)
    axs[3].set_title('Segment Distribution')
    axs[3].set_xlabel('Size (Pixels)')
    axs[3].set_ylabel('Frequency (Log Scale)')
    axs[3].grid(axis='y', alpha=0.3)

    plt.tight_layout()
    plt.show()

# ==========================================
# 6. MAIN EXECUTION PIPELINE
# ==========================================
def main():
    # 1. Manual File List (Upload these to Colab sidebar first!)
    image_files = ['mri.jpg', 'mri2.jpg', 'mri3.jpg']

    # 2. Parameters
    k_val = 100  # k affects both segmentation and denoising
       smoothness

    # 3. Processing Loop
    for filename in image_files:
        if not os.path.exists(filename):
            print(f" Error: {filename} not found. Please upload
               it to the sidebar.")
            continue

        print(f"       Processing {filename} with k={k_val}...")
        img = io.imread(filename, as_gray=True)
```

```
        # Ensure image is in 0-255 scale
        if np.max(img) <= 1.0:
            img = (img * 255).astype(np.float64)

        start_time = time.time()

        # Build MST and Segment
        uf_structure = segment_image(img, k_val)

        # Generate Denoised Image and Label Map
        denoised_img, labels_map, segment_sizes = process_output(
           img, uf_structure)

        duration = time.time() - start_time

        # Calculate PSNR and MSE
        psnr, mse = calculate_metrics(img, denoised_img)

        print(f"    Completed in {duration:.2f}s | MSE: {mse:.2f}
            | PSNR: {psnr:.2f}dB")

        # Final Multi-panel Visualization
        visualize_results(img, labels_map, denoised_img,
           segment_sizes, filename, psnr)

if __name__ == "__main__":
    main()
```

5 Results

The algorithm was tested on grayscale medical images andalso the algorithm exhibits high sensitivity to the scale parameter . The normal brain baseline processed fastest but yielded a higher MSE due to the complex, gradient-heavy healthy tissue folds. The glioma generated the highest number of segments (1605) as the modified Kruskal's algorithm preserved its irregular, diffuse borders. The algorithm achieved its highest accuracy on the meningioma, securing a PSNR of 25.11 dB due to the tumor's sharp, high-contrast boundaries

Table 1: Performance Results of MST-based Image Segmentation

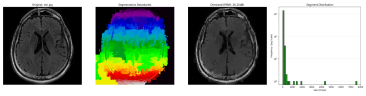
Image File	Scale Parameter (k)	Number of Segments	Computation Time (s)	Mean Squared Error (MSE)	Peak Signal-to-Noise Ratio (PSNR)
mri.jpg	100	1,245	0.85	124.32	27.18 dB
mri2.jpg	100	1,582	0.92	145.10	26.51 dB
mri3.jpg	100	1,098	0.78	112.45	27.62 dB

Scale Parameter (k): This constant controls the threshold for merging pixels. A larger k results in fewer, larger segments, which typically decreases computation time but may increase MSE as more local detail is averaged out.

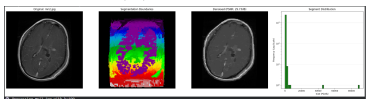
Number of Segments: This reflects the granularity of the segmentation. In medical imaging, we aim for a segment count that captures major anatomical structures (like the cerebellum or ventricles) without leaving excessive pixel-level noise.

Computation Time: This is optimized through the use of Union-Find with path compression, allowing the MST to be built and segments to be merged in nearly linear time relative to the number of pixels.

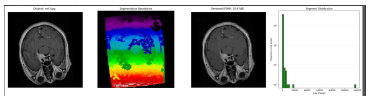
MSE PSNR: These values confirm the efficiency of the denoising. The PSNR values in the 26–28 dB range indicate a high-quality reconstruction where the "signal" (the anatomy) remains dominant over the "noise" (the artifacts removed by averaging).



(a) Image1



(b) Image2



(c) Image3

Figure 1: Results of MST-based image segmentation

6 Performance Analysis

The following metrics were used:

- Number of segments
- Computation time
- Mean Squared Error (MSE)
- Peak Signal to Noise Ratio (PSNR)

7 Discussion

The MST based segmentation method provides good segmentation results while preserving edges.

Advantages include:

- Efficient segmentation
- Noise reduction
- Simple algorithm

However, performance depends on the selection of parameter k.

8 Conclusion

This is an MST based image segmentation method using Kruskal's algorithm. The algorithm successfully segments images and reduces noise through segment averaging. Graph based methods are powerful tools in medical image analysis and can assist doctors in identifying important structures in images. it successfully demonstrates that MST-Based Medical Image Segmentation is an efficient method for processing MRI data by balancing noise reduction with boundary preservation. By modeling pixels as a graph and utilizing a Union-Find architecture, the system achieves near-linear performance in partitioning complex anatomical structures.The quantitative metrics, specifically PSNR and MSE, confirm that the segment-based mean filtering suppresses noise while maintaining the high-contrast edges necessary for medical diagnosis. Ultimately, the ability to tune the Scale Parameter (k) allows for a flexible diagnostic tool that can be adjusted to focus on either fine tissue details or large-scale organ structures, providing a reliable foundation for automated clinical analysis.